Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:

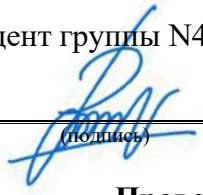
«Web Application Development»

ОТЧЕТ ПО ПРАКТИЧЕСКОМУ ЗАДАНИЮ

«LLM-Chat»

Выполнил:

Перевозник В., студент группы N4150c



(подпись)

Проверил:

Чернышев В. А., преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

CONTENT

INTRODUCTION

The goal of the project is to create an LLM chat using one of the UI strategies.

To achieve this goal this stack is required:

- Python;
- FastAPI;
- PostgreSQL or MongoDB (choose one as the primary database);
- If you use PostgreSQL, schema changes must be applied via database migrations. Alembic is the preferred tool;
- JWT (access/refresh token flow);
- GitHub OAuth;
- Redis.

The primary UI strategy was chosen as SPA (single-page application).

Next will be shown API structure, code structure, UI and database.

1 API STRUCTURE

All backend routes are grouped under the /api prefix and follow a RESTful architecture. The client (SPA frontend) interacts with the server only through JSON payloads and standard HTTP headers.

1.1 Authentication and Session Management (/api/auth)

This group handles registration, authentication, session renewal and logout.

- POST /api/auth/register – registers a new user;
- POST /api/auth/login – authenticates an existing user;
- POST /api/auth/refresh – refreshes an expired access_token;
- POST /api/auth/logout – terminates an active session;
- GET /api/auth/github/login and GET /api/auth/github/callback – implements the OAuth 2.0 flow via Github.

1.2 Chat and Message Management (/api/chats)

This group manages conversations and message history.

- GET /api/chats – returns a list of all chats belonging to the authenticated user;
- POST /api/chats – creates a new conversation session;
- GET /api/chats/{id} – loads a specific chat;
- POST /api/chats/{id}/messages/stream – sends a message and receives the assistant's response.

1.3 System and Frontend Routed

- GET /api/health – lightweight endpoint for infrastructure monitoring;
- GET / - serves the static index.html.

How the API is used:

1. Initial load: browser requests GET / and receives the SPA;

2. Authentication check: frontend reads localStorage, if tokens exist, it silently calls GET /api/chats to restore the session, if the error pops up (401) then it calls /api/auth/refresh, and even if that fails it displays the login form;

3. Register and login: user submits credentials and POST requests /api/auth/{register or login} are called, then the user receives tokens, which are saved to the localStorage, and fetches chat list;

4. Chat interaction: user clicks “Create new chat”, POST /api/chats, receives chat_id and open the chat via GET request /api/chats/{id};

5. Messaging: user types and clicks send, POST /api/chats/{id}/messages/stream and frontend renders are streaming chunks in real-time;

6. Token rotation: on any error 401, frontend pauses UI, calls /api/auth/refresh, updates tokens, retries the original request and resumes UI;

7. Logout: user clicks on the “Logout” button, POST request /api/auth/logout is called, then the storage is cleared and the page reloads.

How does the Authentification work (JWT + Refresh Tokens + Redis):

1. User logs in and backend issues these tokens:
    a. access_token (JWT, 15 mins) that are sent in authorization header;
    b. refresh_token (random UUID, 7 days) that are stored in Redis with TTL;
2. API requests include: authorization: Bearer 'access_token'
3. If access_token expires (401 unauthorized):
    a. frontend calls POST /api/auth/refresh?refresh_token=...;
    b. backend validates token in Redis and issues new pair;
    c. old token is deleted;
4. After logout refresh_token is removed from Redis and frontend clears localStorage.

2 CODE STRUCTURE

The code structure is shown on the schema below.

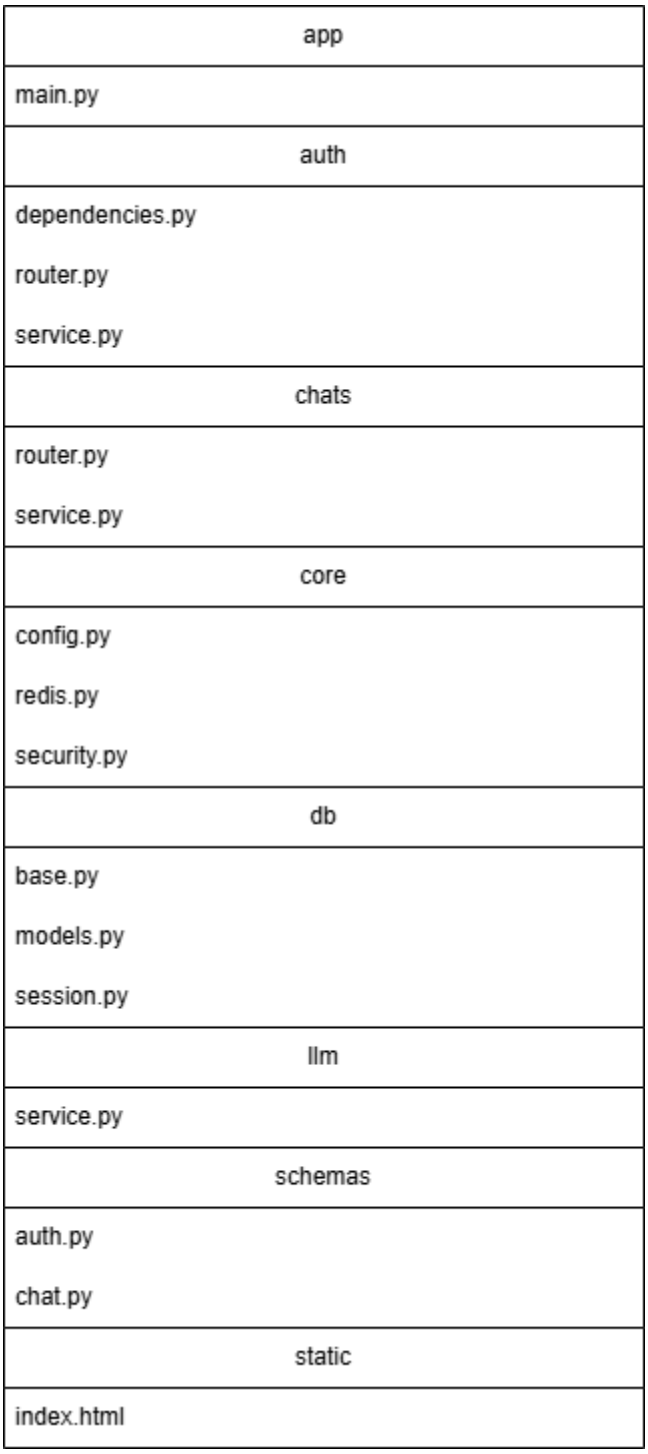


Image 1 – Code structure

3 UI

When entering the localhost:8000 the user is seeing the register and login window.

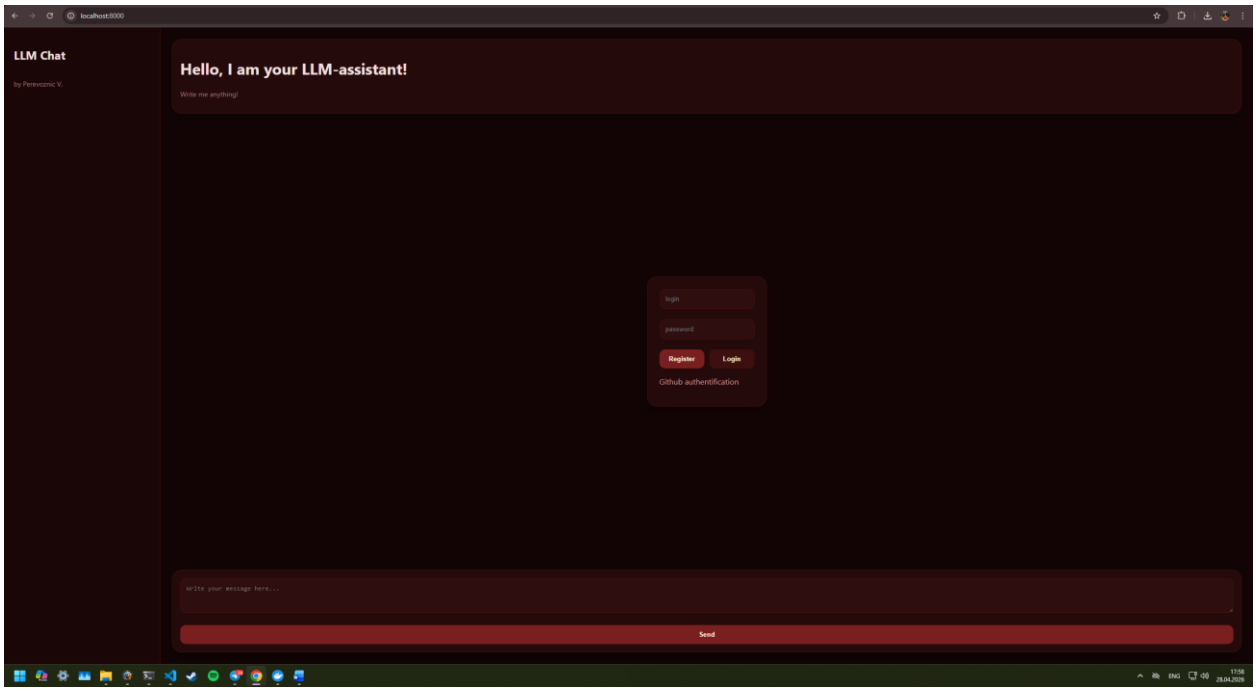


Image 2 – Register/login (pt. 1)

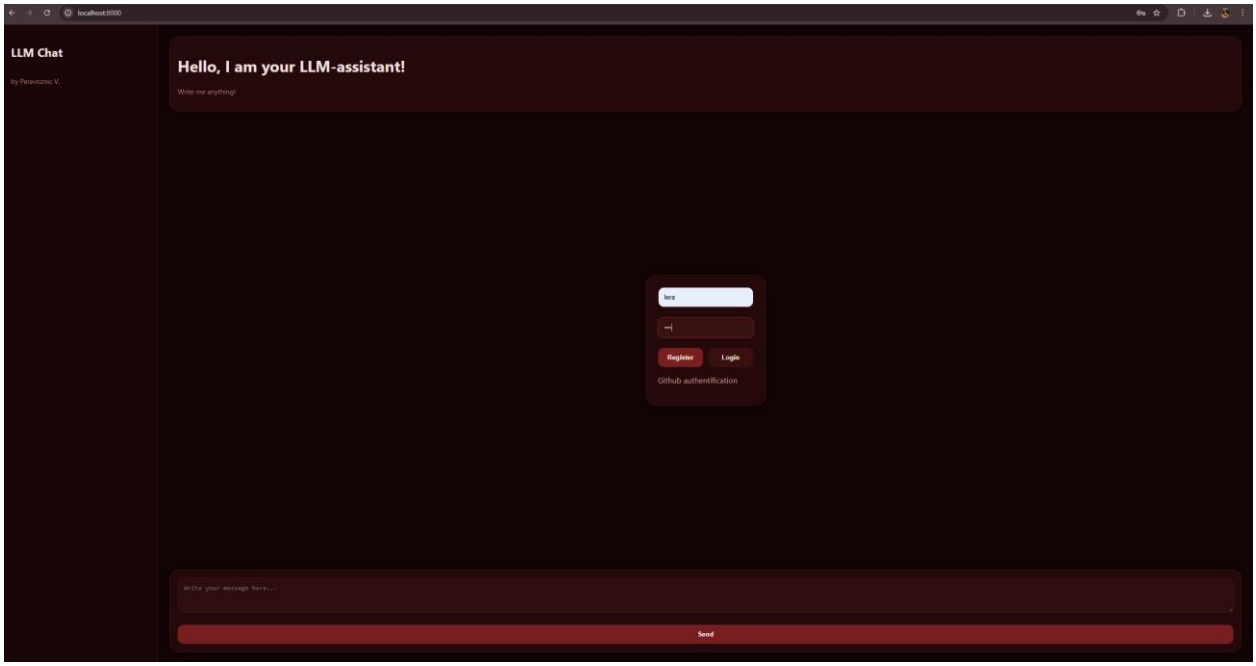


Image 3 – Register/login (pt. 2)

After entering login and password the user sees the chat window.

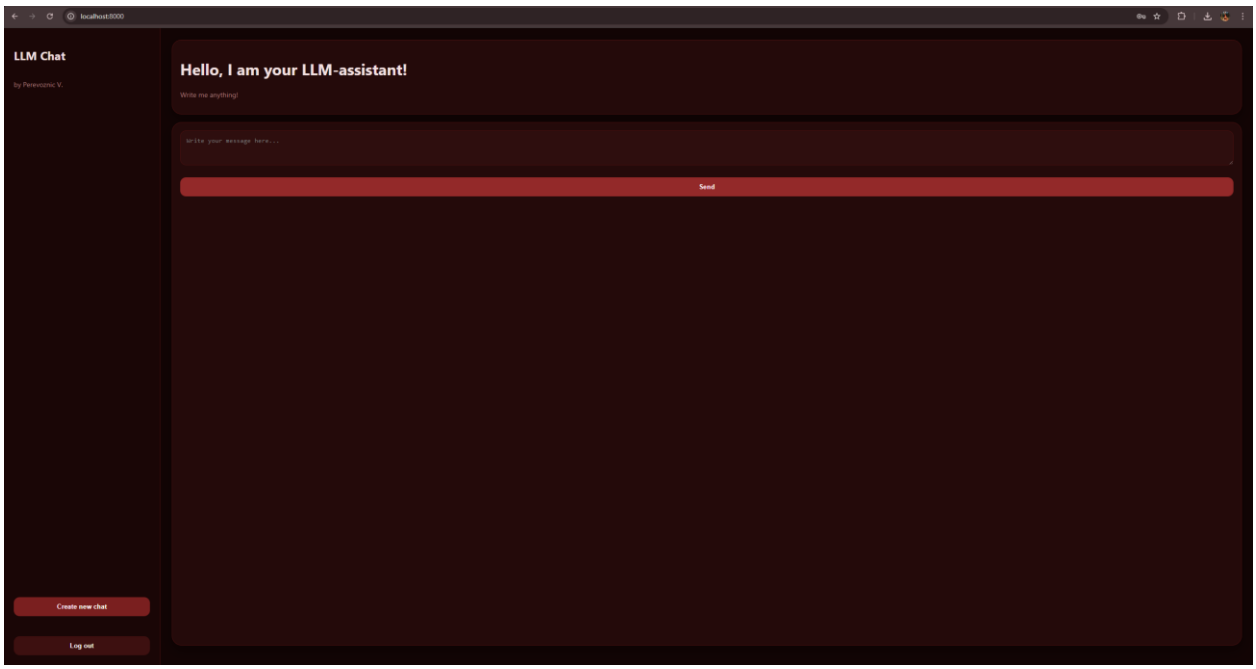


Image 4 – Chat (pt. 1)

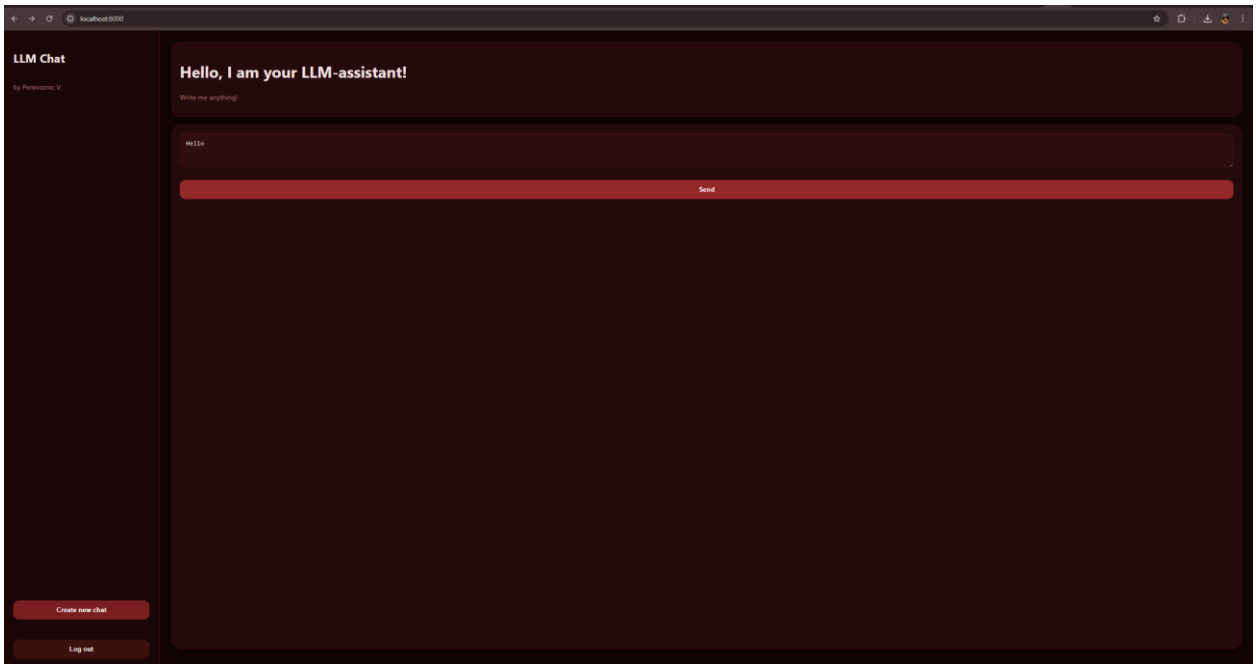


Image 5 – Chat (pt. 2)

4 DATABASE

On the scheme below entity relationships can be seen.

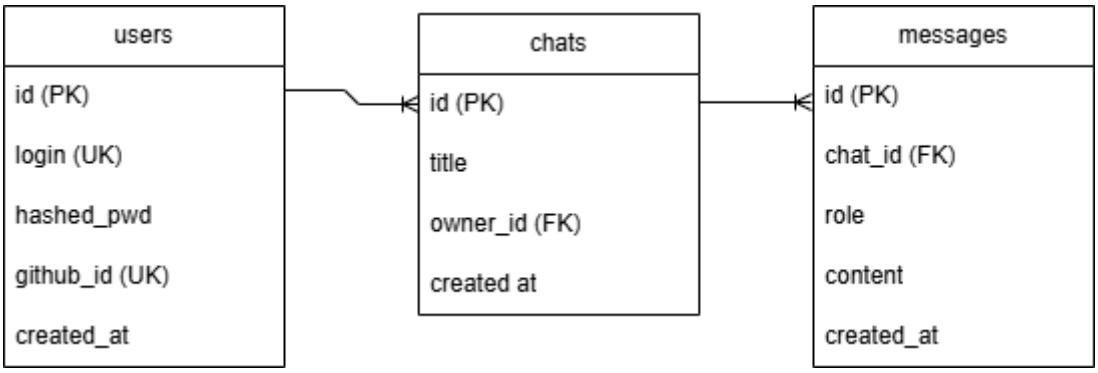


Image 6 – Database ERD

CONCLUSION

This project delivers a functional, secure and scalable LLM-powered chat application.

The system integrates FastAPI backend, a PostgreSQL database managed through Alembic migrations and authentication architecture combining short-lived JWT access token with Redis backend refresh tokens.

The user already can see and try the application by following this instruction:

5. git clone the repository;
6. cd project;
7. copy (or cp for Linux) .env.example .env (change variables in the .env file);
8. Create virtual environment;
9. pip install -r requirements.txt;
10. docker compose up -d;
11. alembic upgrade head;
12. uvicorn app.main:app –reload.